Documentation de validation

Projet Génie Logiciel

Groupe 1, gl06

ZIVKOVIC Lucas, SULLET Benjamin, ZOUGGARI Samy,
TRAORÉ Fié Victor, HURSTEL Mathis

21 Janvier 2025

Grenoble INP - Ensimag
Génie Logiciel 2024-2025

Sommaire

Contents

1 Importance et objectif de la validation dans le projet **2**

2 Approche globale de la validation **2**

3 Phase Syntaxique **3**
3.1 Tests du Lexer (`DecaLexer.g4`) 3
3.2 Tests de la Grammaire Syntaxique 3
3.3 Approche Globale 4

4 Validation de la Phase Contextuelle **4**
4.1 Approche et Méthodologie 5
4.1.1 Conditions des Structures de Contrôle 5
4.1.2 Arguments des Instructions `print` et Variantes 5
4.1.3 Comparaisons et Compatibilité des Types 5
4.1.4 Assignations 5
4.1.5 Opérations Arithmétiques 6
4.2 Résumé des Tests 6

5 Validation de la génération de code **7**

6 Automatisation **8**

7 Résultats **9**

1 Importance et objectif de la validation dans le projet

En tant que futurs ingénieurs, il est essentiel d'être sensibilisés à l'importance de produire des outils de qualité. Ce projet a été une excellente opportunité pour nous d'adopter une démarche qualité rigoureuse, avec pour ambition de développer un compilateur répondant aux exigences du cahier des charges tout en visant un code "zéro défaut". Nous avons pleinement intégré que garantir la fiabilité d'un produit logiciel nécessite de consacrer des efforts significatifs à sa validation et à son contrôle.

Dans cette optique, nous avons mis en place des méthodes structurées pour assurer la conformité et la robustesse de notre compilateur. Cela s'est traduit par une stratégie de développemnt sous la forme de TDD (Test Driven Development) ainsi qu'une large base de tests couvrant les différentes étapes de la compilation (analyse syntaxique, contextuelle et génération de code), par des tests de non-régression, et par leur automatisation pour en faciliter l'exécution. Ces démarches nous ont permis de détecter rapidement les erreurs et de garantir la stabilité du produit au fil des itérations.

Par ailleurs, nous avons complété ces efforts techniques par des pratiques de revue de code. Ces revues nous ont permis d'améliorer la qualité de notre code non seulement en termes de correction, mais également en termes de lisibilité, maintenabilité et respect des bonnes pratiques de développement.

Ainsi, cette documentation vise à présenter en détail les démarches de validation que nous avons mises en œuvre, leur impact sur le développement du projet, et les résultats obtenus. Nous espérons qu'elle témoignera de notre engagement envers la qualité et de notre volonté de produire un outil fiable et performant.

Ce projet est divisé en trois phases principales :

1. La phase syntaxique ;
2. La phase contextuelle ;
3. La phase assembleur.

Pour assurer la validité de chaque phase, une série de tests a été réalisée. Ces tests couvrent l'ensemble des étapes du projet et garantissent que chaque composant fonctionne correctement.

2 Approche globale de la validation

Comme mentionné plus haut la haute qualité exigé pour ce projet necessitait de gros efforts de validation. La première démarche adoptée à été de vouloir créer une très grandes base de test, la plus exaustive possible afin de nous permettre d'identifier si notre compilateur n'acceptait bien que ce qui était permis par le langage, et de détecter de potentielles erreurs que nous avions pu faire. C'est

pour cela que sur notre planing initial nous avions prévu de passer pratiquement autant de temps sur le dévelopement pur du compilateur que sur les phases de tests. Mais la validation n'est pas uniquement passée par la création de nombreux tests, nous avons aussi voulu les automatiser afin de gagner du temps et d'éviter au plus possible l'erreur humaine. Un autre point clef de notre stratégie a aussi été le TDD, que nous avons particulièrement mis en place lors de la phase de génération de code. De plus nous avons tenus à ce que les personnes qui rédigent les tests ne soient pas les mêmes qui aient implémentés la fonctionnalité testée. Ceci est important pour maximiser nos chances de détecter d'eventuelles erreurs et augmenter la couvertures de nos tests. Durant ce projet nous avons concu différents types de tests, à commencer par des tests unitaires durant les deux première étapes de la compilation. Ensuite pour la génération de code nous avons directement opté pour des tests fonctionnels visant à identifier si toutes les étapes de la chaîne de compilation s'exécuter bien une fois mises bout à bout. Enfin, les derniers types de tests que nous avons mis en place sont des tests de non regression que nous avons essayé de coupler avec une pipeline gitlab qui serait vérifiée à chaque push d'un utilisateur (malheureusement il s'avère que le pipeline ne soit pas fonctionnelle).

3 Phase Syntaxique

La phase syntaxique se concentre sur la vérification de la conformité des programmes aux règles de grammaire définies pour le langage Deca. Plusieurs étapes de tests ont été réalisées:

3.1 Tests du Lexer (`DecaLexer.g4`)

Nous avons commencé par tester l'ensemble des mots définis dans le fichier `DecaLexer.g4`, afin de vérifier que chaque mot du langage est correctement reconnu. Cela inclut :

- Les mots-clés;
- Les opérateurs;
- Les symboles spécifiques au langage Deca.

3.2 Tests de la Grammaire Syntaxique

Pour valider la grammaire syntaxique (hors-contexte), nous avons effectué deux types de tests:

1. **Tests invalides** : Ces tests utilisent des programmes contenant des erreurs syntaxiques. Ils permettent de vérifier que :
    - Le compilateur détecte correctement les erreurs;

- Les messages d'erreur contiennent les informations nécessaires (nom du fichier, numéro de ligne, et symbole problématique, notamment si un mot est absent du lexer).



Figure 1: /Users/lucaszivkovic/ensimag/GL/Projet$_GL/src/test/deca/syntax/invalid/bad_synt1.deca$: $4 : 0 : mismatchedinput' >' expecting\{' false',' if', ...\}$

2. **Tests valides** : Ces tests s'assurent que les programmes conformes à la grammaire syntaxique sont bien acceptés par le compilateur.

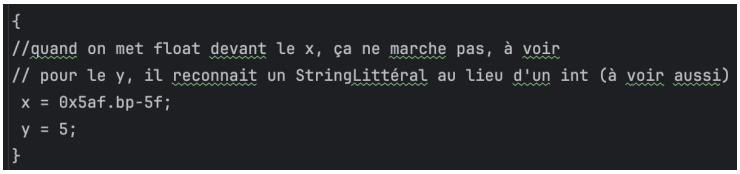


Figure 2: decac -p: { x = 0x1.6becp5; y = 5; }

3.3 Approche Globale

Pour la partie syntaxique hors-contexte, nous avons suivi une approche systématique:

- Dans un premier temps, nous avons vérifié que les programmes syntaxiquement incorrects sont rejetés, en identifiant précisément leurs erreurs avec l'emplacement du fichier, la postion de l'erreur dans le fichier, ainsi que son erreur;
- Ensuite, nous avons validé que tous les programmes corrects (du point de vue syntaxique) sont bien acceptés.

Cette démarche garantit la fiabilité de la phase syntaxique et fournit une base solide pour les phases contextuelle et assembleur.

4 Validation de la Phase Contextuelle

La deuxième phase de validation concerne la syntaxe contextuelle. Cette étape consiste à vérifier que les programmes Deca respectent les contraintes contextuelles du langage. Pour cela, nous avons mis en place une série de tests afin de garantir que:

- Les programmes Deca invalides sur le plan contextuel sont bien détectés comme tels;
- Les programmes Deca valides sur le plan contextuel ne posent aucun problème à la compilation.

4.1 Approche et Méthodologie

La validation contextuelle repose sur une analyse approfondie des règles du langage. Voici les principaux aspects vérifiés:

4.1.1 Conditions des Structures de Contrôle

Dans les structures de contrôle telles que `if`, `else if` et `while`, nous avons vérifié que les conditions sont bien des expressions de type `boolean`. Une condition incorrecte doit entraîner une erreur contextuelle.

4.1.2 Arguments des Instructions `print` et Variantes

Pour les instructions d'affichage (`print`, `println`, `printx`, `printlnx`), les types des arguments doivent être conformes. Nous vérifions que ces arguments appartiennent à l'un des types suivants:

- `string` ;
- `int` ;
- `float`.

Toute tentative d'utiliser un type non autorisé dans ces instructions déclenche une erreur contextuelle.

4.1.3 Comparaisons et Compatibilité des Types

Lors des comparaisons (`==`, `!=`, `<`, `>`, `<=`, `>=`), nous avons vérifié que les types des opérandes sont identiques ou compatibles (via un cast implicite ou explicite). Une erreur est levée si les types ne sont pas compatibles.

4.1.4 Assignations

Dans les expressions d'assignation, nous avons contrôlé que le type de l'expression située à droite correspond au type de la variable située à gauche, ou qu'il est possible de réaliser un cast valide. Les règles de cast sont définies comme suit:

- Dans le cas de classes, le type de l'expression à droite doit être une sous-classe du type de la variable;
- Pour les types primitifs `int` et `float`, un cast implicite ou explicite est autorisé.

4.1.5 Opérations Arithmétiques

Pour les opérations arithmétiques (`+`, `-`, `*`, `/`, `%`), nous avons vérifié que les types des opérandes respectent les règles du langage:

- Les deux opérandes doivent être du même type (`int` ou `float`);
- Si les types diffèrent, un cast doit être possible pour permettre l'opération.

et de la même manière pour le return dans les methodes.

4.2 Résumé des Tests

Les tests ont permis de valider les points suivants:

- Les erreurs contextuelles sont correctement détectées, avec des messages explicites pour aider à leur résolution;

```
class A {
    int x = 1;

    float getX(){
        return x;
    }
}
```

Figure 3: /Users/lucaszivkovic/ensimag/GL/Projet$_GL$/src/$test$/$deca$/$context$/$invalid$/$bad_return_type.deca$: 12 : 8 : $lereturnn'estpasdubontype$

- Les programmes valides sont compilés sans aucune erreur contextuelle.

```
{
boolean a = true;
int i=0;
if (a){
    while(i<10){
        i=i+1;
    }
    println(i);
}
}
```

Figure 4: decac -v:

Grâce à ces vérifications, nous avons assuré la robustesse de la phase syntaxique contextuelle, garantissant que le code respecte les règles sémantiques du langage Deca.

5 Validation de la génération de code

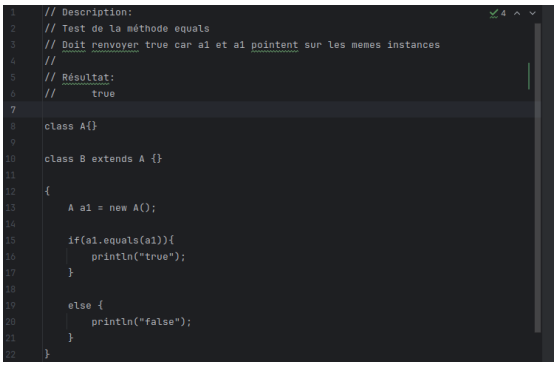
Il est maintenant temps de se pencher sur la validation de la dernière étape de la chaîne de compilation de notre outils : la génération de code. L'objectif de cette phase est double :

- S'assurer que le code assembleur généré soit bien exécuté par `ima` et que le résultat renvoyé soit identique à celui attendu ;
- Vérifier que si une erreur sémantique est présente elle soit bien levée lors de l'exécution (division par 0);

Pour accomplir ces deux objectifs, nous avons eu recours au TDD dans un premier temps, et nous avons adopter une approche incrémentale. Au début nos tests ne faisait que de simples opérations arithmétiques (voir Figure 5), puis des opération plus complexes, et ensuite nous avons ajouter des classes sans méthodes puis enfin des classes (voir Figure 6)avec des méthodes de plus en plus pointues. Cet double approche à permis une bonne parallélisation du travail et ainsi une bonne efficacité. A chaque incrément, une fois que nous pensions que la fonctionnalité souhaitée était fonctionnelle, une personne qui n'avait pas codé cette fonctionnalité venait enrichir la base de tests déjà présente, que ce soit des tests correctes ou incorrectes.

```
1   // Description :
2   //      tests d'une division par zéro entre int
3   //
4   // Resultats :
5   //      Erreur: Division by zero
6
7
8   {
9       int a = 63;
10      int b = 0;
11      int z = a / b;
12
13      println("", a, " / ", b, " = ", z);
14  }
```

Figure 5: Fichier intdivisionbyzero.deca



```
// Description:
// Test de la méthode equals
// Doit renvoyer true car a1 et a1 pointent sur les memes instances
//
// Résultat:
//      true

class A{}

class B extends A {}

{
    A a1 = new A();

    if(a1.equals(a1)){
        println("true");
    }

    else {
        println("false");
    }
}
```

Figure 6: Fichier goodEquals3.deca

6 Automatisation

Une fois que notre base de test nous emblait assez importante, nous avons jugé intéressant de les automatiser. Cette automatisation a plusieurs but, tester les nombreux fichiers Deca que nous avons beaucoup plus rapidement, mais aussi supprimer les potentielles erreurs humaines lorsque l'on vérifie si le tests renvoie bien ce que l'on souhaite. Pour ce faire nous avons écrit différents script bash qui sont situé dans le dossier `src/test/script/` et dans le sous-dossier `launchers/`. Dans ces dossier on retrouve plusieurs fichier bash, dont un certain nombre nous était déjà fourni comme `common-test.sh`. Ce dernier à joué un rôle important dans notre processus de validation puisqu'il était nécessaire qu'il renvoie un message positif pour vérifier que les fonctionnalités de bases soient bien fonctionnelles. Parmis les scripts que nous avons écrit nous retrouvons :

- `non_regression` : qui nous permettait de tester la non regression de notre compilateur. Pour se faire, une fois que les fonctionnalités importantes étéaient implémentées, nous lancions notre compilateur sur tous nos fichiers de tests deca et nous stockions les résultats dans un dossier `resultats/`. Puis lors de l'implémentation d'une nouvelle fonctionnalité, nous executions le script `non_regression` du dossier `launchers` qui allait lui aussi compiler tous les .deca et comparé la sortie avec l'ancienne sortie qui était stockée dans le dossier `resultats`. Si il n'y avait pas de différences, c'est que rien n'a été cassé, mais en cas de différences ils nous fallait aller voir où et pourquoi ;

- `options_test.sh` : qui est un test visant à tester l'option -p du compilateur. ce script bash compile tous les fichiers deca avec l'option -p, et on vérifie que le code deca décompilé est bien identique à celui donné en entrée. Malheureusement ce test n'est pas complétement utilisable puisque les structure de contrôle if else, ne sont pas décompilées à l'identique.

- `test-synt.sh`, `test-cont.sh` et `test-Codegen.sh` (rajoutés après la date

de rendu du projet) qui sont scripts assez simple qui permettent simplement de compiler respectivement tous les fichiers .deca des dossiers `src/test/syntax/`, `src/test/context/` et `src/test/codegen/`.

7 Résultats

La validation de notre compilateur a été une étape cruciale pour garantir la fiabilité et la qualité du produit final. Grâce à une approche rigoureuse et structurée, nous avons élaboré une stratégie de validation globale qui a couvert les différentes phases de compilation : syntaxique, contextuelle et génération de code.

En mettant l'accent sur le TDD, une base de tests exhaustive (total de 320 tests), et une automatisation avancée, nous avons pu détecter rapidement des erreurs et assurer une évolution stable de notre projet. Ces efforts ont permis d'atteindre une couverture de code de 60% selon l'analyse Jacoco. Bien que ce résultat témoigne d'un travail conséquent, il met également en évidence que l'intégration de Jacoco plus tôt dans le processus aurait pu améliorer encore davantage notre couverture et nos capacités de validation.

Enfin, ce projet nous a permis de prendre conscience des défis liés à la validation logicielle dans un cadre professionnel. En tant que futurs ingénieurs, cette expérience a solidifié notre compréhension de l'importance d'une démarche qualité robuste et nous a donné les outils nécessaires pour aborder des projets futurs avec encore plus d'exigence et de rigueur.

En somme, bien que des améliorations soient toujours possibles, notamment en matière de couverture des tests (ou pour gérer le langage Deca complet), nous sommes satisfaits des résultats obtenus et assez confiants dans la robustesse de notre compilateur.